



National Technical University of Athens

School of Electrical and Computer Engineering

8th Semester

VLSI Digital Systems

3rd Laboratory Exercise

Dimitris Vatalas

(Student ID: 03121093)

Aristotelis Georgios Dritsas

(Student ID: 03122127)

Athens, March 2026

Contents

Introduction	2
MAC: Multiply-Accumulate Unit	2
ROM	3
RAM	5
Control Unit	6
Complete Circuit – FIR Filter	7

Introduction

A FIR filter is generally described by the following equation:

$$y[n] = \sum_{k=0}^M h[k]x[n-k] = h[0]x[n] + h[1]x[n-1] + \dots + h[M]x[n-M]$$

where:

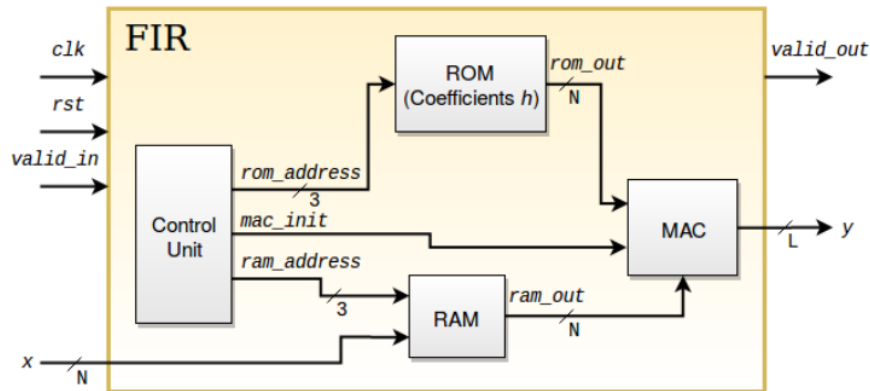
M: the filter order

y[n]: the discrete output at time instant n

h[k]: the k -th filter coefficient

x[n]: the input value at time instant n

In this laboratory exercise, according to the specifications, we will implement an FIR filter with 8 taps. According to the equation above, this corresponds to a filter of order $M = 7$, i.e., a filter with 8 terms in total. The final circuit that will be implemented is shown below:



MAC: Multiply-Accumulate Unit

This structural component of the circuit is responsible for calculating the output values y by performing the following operation:

$$a \leftarrow a + b \cdot x$$

The MAC (Multiply-Accumulate) unit receives the filter coefficients and the successive values of the input signal as inputs, while producing the corresponding output sample y . In addition, it includes a control signal that is used to indicate the beginning of the calculation of a new output value and, consequently, the initialization of the MAC unit.

Particular attention must be paid to the required width of the filter output y . More specifically, each output sample must be represented using $2N + \log_2 N$ bits, since it is obtained from the sum of N products, where each product corresponds to the multiplication of two N -bit numbers and therefore has a width of up to $2N$ bits.

The following VHDL code implements the corresponding unit using a behavioral architecture.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.std_logic_unsigned.all;

entity mac is
    Port (
```

```

    a: in std_logic_vector (7 downto 0);
    b: in std_logic_vector (7 downto 0);
    clk: in std_logic;
    mac_init: in std_logic;

    y: out std_logic_vector (18 downto 0)
);
end mac;

architecture Behavioral of mac is

    signal accumulator: std_logic_vector(18 downto 0) := (others => '0');

begin

    process(clk)
    begin
        if rising_edge(clk) then
            if mac_init = '1' then
                accumulator <= "000" & (a * b);
            else
                accumulator <= accumulator + (a * b);
            end if;
        end if;

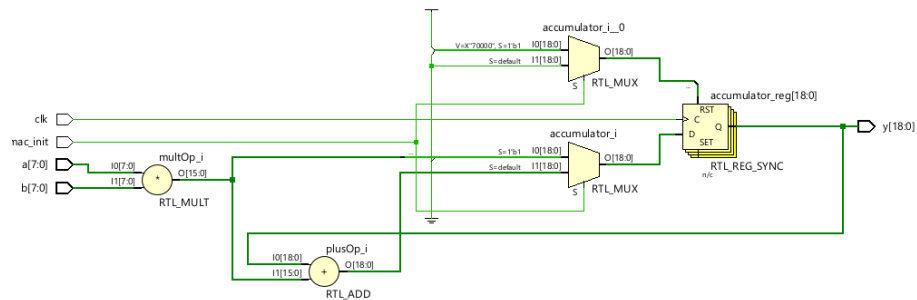
    end process;

    y <= accumulator;

end Behavioral;

```

Using the elaborated design, we obtain the following result:



ROM

The ROM memory is used to store the constant filter coefficients h . It receives an address as input and returns the corresponding stored value at its output, but only when the enable signal is set to logic 1. According to the specifications, the stored filter coefficients are:

$$h = \{1, 2, 3, 4, 5, 6, 7, 8\}$$

The following VHDL code describes the operation of the ROM:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_unsigned.all;

entity mlab_rom is
    generic (
        coeff_width : integer := 8 --- coefficient width (bits)
    );
    Port (
        clk : in STD_LOGIC;
        en : in STD_LOGIC; --- operation enable
        addr : in STD_LOGIC_VECTOR (2 downto 0); -- memory address
        rom_out : out STD_LOGIC_VECTOR (coeff_width-1 downto 0); -- output data
        rst : in std_logic
    );
end mlab_rom;

architecture Behavioral of mlab_rom is

    type rom_type is array (7 downto 0) of std_logic_vector (coeff_width-1 downto 0);
    signal rom : rom_type:= ("00001000", "00000111", "00000110", "00000101", "00000100",
        "00000011", "00000010", "00000001"); -- ROM initialization with user data

    signal rdata : std_logic_vector(coeff_width-1 downto 0) := (others => '0');
begin

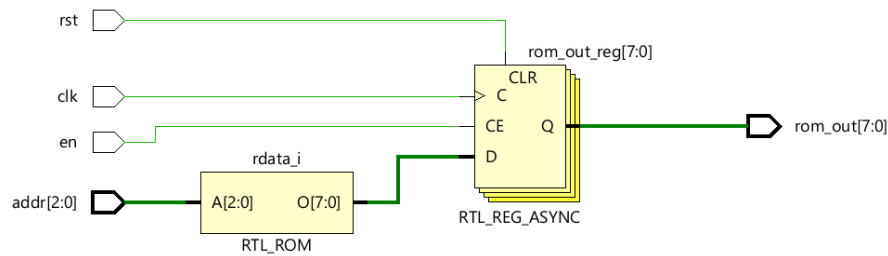
    rdata <= rom(conv_integer(addr));

    process (clk, rst)
    begin
        if rst = '1' then
            rom_out <= (others => '0');
        elsif rising_edge(clk) then
            if en = '1' then
                rom_out <= rdata;
            end if;
        end if;
    end process;

end Behavioral;

```

Using the elaborated design, we obtain:



RAM

The RAM memory is used to store the current value of the input signal, as well as a history of the seven previous samples, which are required for calculating the next output value of the filter. During a read operation, the RAM receives an address as input and returns the corresponding stored value.

During a write operation, the most recent signal value is placed at the first memory location, while the previously stored values are shifted by one position, resulting in the oldest value being discarded.

It is important to note that during the write operation, the RAM has been appropriately implemented so that the newly written value appears immediately at the output at the same time as it is stored. This avoids the introduction of a one-clock-cycle delay that would occur if the read operation followed the write operation.

The following VHDL code describes the operation of the RAM:

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;

entity mlab_ram is
    generic (
        data_width : integer := 8 -- width of data (bits)
    );
    port (
        clk : in std_logic;
        we : in std_logic; -- memory write enable
        en : in std_logic; -- operation enable
        addr : in std_logic_vector(2 downto 0); -- memory address
        di : in std_logic_vector(data_width-1 downto 0); -- input data
        rst : in std_logic;
        do : out std_logic_vector(data_width-1 downto 0); -- output data
    end mlab_ram;

    architecture Behavioral of mlab_ram is

        type ram_type is array (7 downto 0) of std_logic_vector (data_width-1 downto 0);
        signal RAM : ram_type := (others => (others => '0'));
        signal tmp: std_logic;

    begin

        process (clk, rst)
        begin

            if rst = '1' then
                RAM <= (others => (others => '0'));
                do <= (others => '0');

            elsif clk'event and clk = '1' then
                if en = '1' then
                    if we = '1' then -- write operation
                        for i in 7 downto 1 loop
                            RAM(i) <= RAM(i-1);
                        end loop;
                        RAM(0) <= di;
                        do <= di;
                    else -- read operation
                        do <= RAM(conv_integer(addr));
                    end if;
                end if;
            end if;
        end process;
    end architecture Behavioral;
end mlab_ram;
```

```

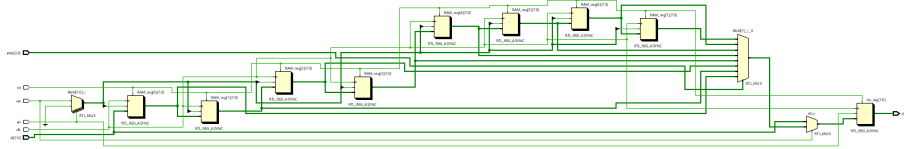
        end if;

    end process;

end Behavioral;

```

Using the elaborated design, we obtain:



Control Unit

The Control Unit is responsible for synchronizing the system, providing the appropriate addresses to the RAM and ROM at each clock cycle, as well as the signal that activates the initialization of the MAC unit. In this way, it ensures the proper coordination of all the individual components of the system. The following VHDL code implements the Control Unit:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.std_logic_unsigned.all;

entity control_unit is
    Port(
        valid_in: in std_logic;
        clk: in std_logic;
        rst: in std_logic;

        en: out std_logic;
        we: out std_logic;
        mac_init: out std_logic;
        rom_addr: out std_logic_vector(2 downto 0);
        ram_addr: out std_logic_vector(2 downto 0);
        valid_out: out std_logic
    );
end control_unit;

architecture Behavioral of control_unit is

    signal count: std_logic_vector(2 downto 0) := "000";
    signal enabled_flag: std_logic := '0';

begin

    process(clk, rst, valid_in)
    begin
        if rst = '1' then
            count <= "000";
            mac_init <= '1';
            valid_out <= '0';

```


Control Unit, the final architecture of the system is essentially limited to appropriately connecting the individual submodules.

More specifically, the ROM, RAM, and MAC units are interconnected according to the original schematic presented in the introduction. At the same time, they are connected to the external inputs and outputs of the complete circuit, which is now treated as a single structural component, namely the FIR filter.

The circuit receives new input data every 8 clock cycles, accompanied by a valid signal indicating that the input value is ready for processing. Correspondingly, the filter produces a new output sample every 8 clock cycles, accompanied by a valid signal that distinguishes valid results from intermediate values.

The following VHDL code describes the operation of the FIR filter using a structural architecture:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.std_logic_unsigned.all;

entity fir is
    Port (
        clk: in std_logic;
        rst: in std_logic;
        valid_in: in std_logic;
        x: in std_logic_vector (7 downto 0);

        valid_out: out std_logic;
        y: out std_logic_vector (18 downto 0)
    );
end fir;

architecture Behavioral of fir is

    component control_unit is
        Port (
            valid_in: in std_logic;
            clk: in std_logic;
            rst: in std_logic;

            en: out std_logic;
            we: out std_logic;
            mac_init: out std_logic;
            rom_addr: out std_logic_vector(2 downto 0);
            ram_addr: out std_logic_vector(2 downto 0);
            valid_out: out std_logic
        );
    end component;

    component mac is
        Port (
            a: in std_logic_vector (7 downto 0);
            b: in std_logic_vector (7 downto 0);
            clk: in std_logic;
            mac_init: in std_logic;

            y: out std_logic_vector (18 downto 0)
        );
    end component;

    component mlab_ram is
```

```

        generic (
            data_width : integer :=8 -- width of data (bits)
        );
    Port (
        clk : in std_logic;
        we : in std_logic; -- memory write enable
        en : in std_logic; -- operation enable
        addr : in std_logic_vector(2 downto 0); -- memory address
        di : in std_logic_vector(7 downto 0); -- input data
        rst : in std_logic;

        do : out std_logic_vector(7 downto 0) -- output data
    );
end component;

component mlab_rom is
    generic (
        coeff_width : integer :=8 --- coefficient width (bits)
    );
    Port (
        clk : in STD_LOGIC;
        en : in STD_LOGIC; --- operation enable
        addr : in STD_LOGIC_VECTOR (2 downto 0); -- memory address

        rom_out : out STD_LOGIC_VECTOR (coeff_width-1 downto 0) ; -- output data
        rst : in std_logic
    );
end component;

signal rom_addr, ram_addr: std_logic_vector(2 downto 0):= (others => '0');
signal rom_data, ram_data: std_logic_vector(7 downto 0):= (others => '0');
signal rom_out, ram_out: std_logic_vector (7 downto 0) := (others => '0');
signal mac_init, en, we: std_logic;

begin
    rom_out <= rom_data when en = '1' else (others => '0');
    ram_out <= ram_data when en = '1' else (others => '0');

    cu: control_unit port map(
        clk => clk,
        rst => rst,
        en => en,
        we => we,
        valid_in => valid_in,
        rom_addr => rom_addr,
        ram_addr => ram_addr,
        mac_init => mac_init,
        valid_out => valid_out
    );

    rom: mlab_rom port map(
        clk => clk,
        en => en,
        addr => rom_addr,
        rom_out => rom_data,

```

```

        rst=>rst
    );

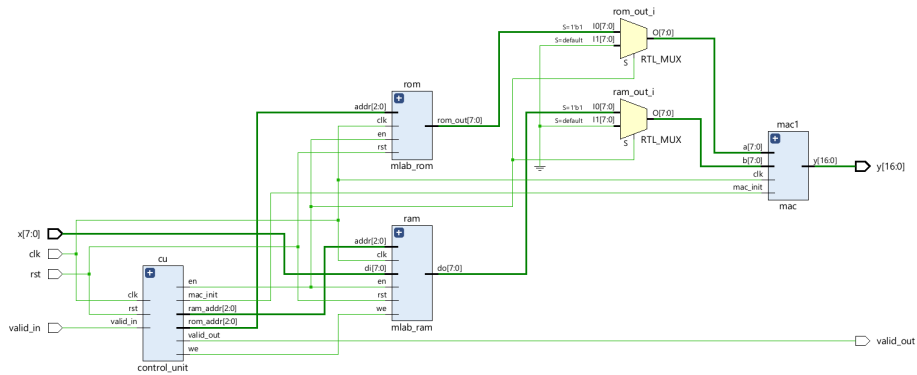
    ram: mlab_ram port map(
        clk => clk,
        we => we,
        en => en,
        addr => ram_addr,
        di => x,
        rst => rst,
        do => ram_data
    );

    mac1: mac port map(
        clk => clk,
        a => rom_out,
        b => ram_out,
        mac_init => mac_init,
        y => y
    );

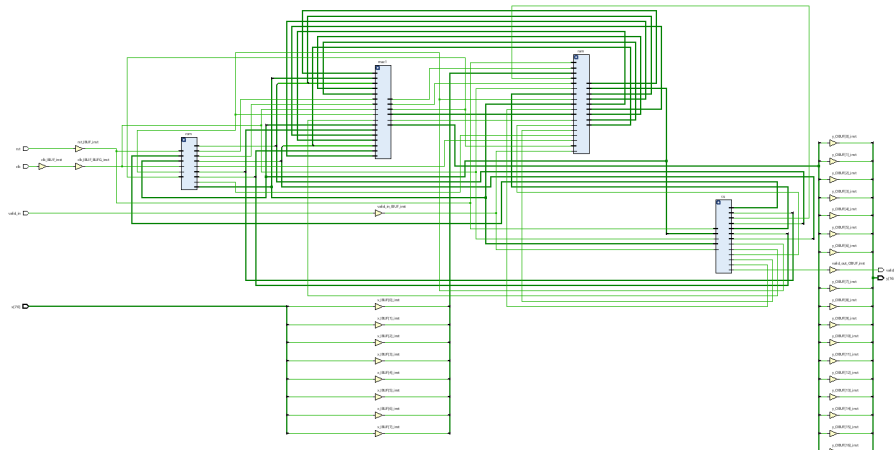
end Behavioral;

```

Using the elaborated design, we obtain:

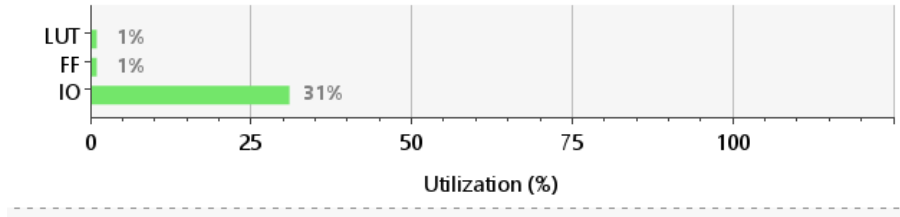


Using the post-implementation design, we obtain:

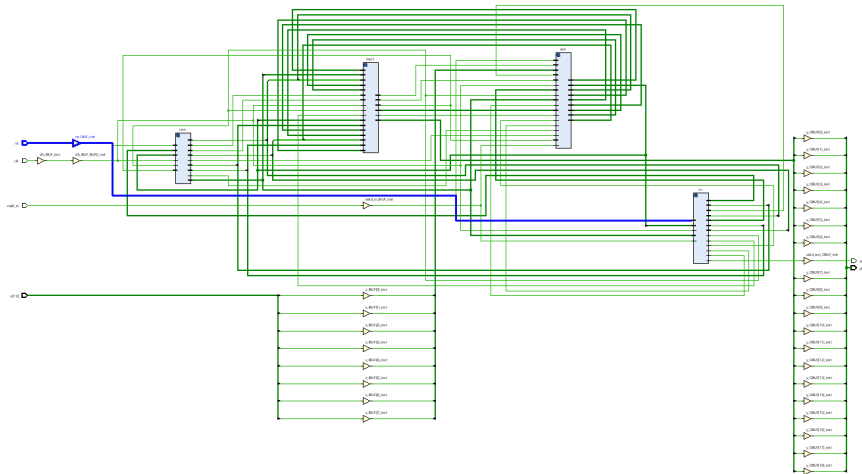


The resources used for the implementation of the circuit are presented below, as obtained through the *Utilization* report of the synthesized design.

Resource	Utilization	Available	Utilization %
LUT	91	17600	0.52
FF	101	35200	0.29
IO	31	100	31.00



The following critical path is identified using the timing summary:



Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clk
Path 21	∞	3	3	82	rst	cu/enabled_flag_reg/D	3.749	1.983	1.766	∞	input port clock	
Path 22	∞	1	1	82	rst	cu/enabled_flag_reg/CLR	2.169	0.961	1.209	∞	input port clock	

We observe that the critical path has a delay of 3.749 ns, which corresponds to a maximum operating frequency of 266.95 MHz.

The testbench developed to verify the correct operation of the circuit is presented below. The simulation results are then presented and discussed:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity fir_tb is
end fir_tb;

```

architecture behavior of fir_tb is

```
component fir
  port (
    clk      : in  std_logic;
    rst      : in  std_logic;
    valid_in  : in  std_logic;
    x        : in  std_logic_vector(7 downto 0);
    valid_out : out std_logic;
    y        : out std_logic_vector(16 downto 0)
  );
end component;

signal clk      : std_logic := '0';
signal rst      : std_logic := '1';
signal valid_in  : std_logic := '0';
signal x        : std_logic_vector(7 downto 0) := (others => '0');
signal valid_out : std_logic;
signal y        : std_logic_vector(16 downto 0);

type input_type is array (0 to 31) of std_logic_vector(7 downto 0);

signal input : input_type := (
  "11010000", "11100111", "01000000", "11101001",
  "10100001", "00011000", "01000111", "10001100",
  "11110101", "11110111", "00101000", "11111000",
  "11110101", "01111100", "11001100", "00100100",
  "01101011", "11101010", "11001010", "11110101",
  "00000000", "00000000", "00000000", "00000000",
  "00000000", "00000000", "00000000", "00000000",
  "00000000", "00000000", "00000000", "00000000"
);

constant CLK_PERIOD : time := 10 ns;

begin

  uut: fir
    port map (
      clk      => clk,
      rst      => rst,
      valid_in  => valid_in,
      x        => x,
      valid_out => valid_out,
      y        => y
    );

  -- Clock generation
  clk_process : process
  begin
    while now < CLK_PERIOD * 1000 loop
      clk <= '0';
      wait for CLK_PERIOD / 2;
    end loop;
  end process;
end;
```

```

        clk <= '1';
        wait for CLK_PERIOD / 2;
    end loop;
    wait;
end process;

-- Stimulus process
stim_process : process
begin
    -- Reset sequence
    rst <= '1';
    valid_in <= '0';
    wait for 10 * CLK_PERIOD ;

    rst <= '0';
    wait for 2 * CLK_PERIOD;

    -- Calculate again
    for i in 0 to 31 loop
        valid_in <= '1';
        x <= input(i);
        wait for CLK_PERIOD;

        valid_in <= '0';
        wait for 7 * CLK_PERIOD;
    end loop;

    wait;
end process;
end behavior;

```

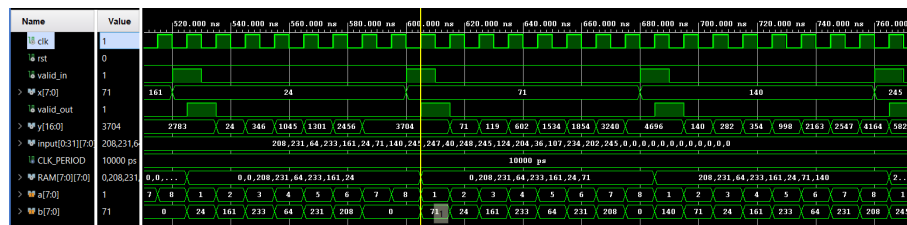
Initially, a reset signal is applied to the circuit, after which the following input values are provided:

$$x = \{208, 231, 64, 233, 161, 24, 71, 140, 245, 247, 40, 248, 245, 124, 204, 36, 107, 234, 202, 245, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0\}$$

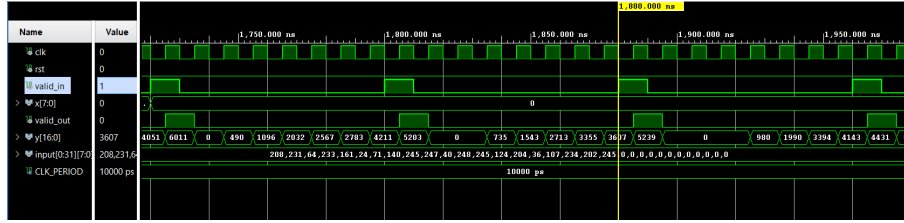
The expected output is calculated to be:

$$y = \{208, 647, 1150, 1886, 2783, 3704, 4696, 5828, 5333, 4670, 5319, 4631, 4603, 5771, 6696, 6965, 6256, 5518, 6598, 6011, 5203, 5231\}$$

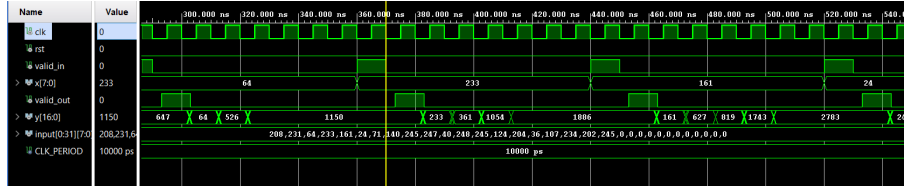
We run the behavioral simulation and obtain the results shown in the figure below. We observe that the filter produces the expected output values. The `valid_out` signal, which indicates the validity of the output, is correctly synchronized with the corresponding valid values:



Next, we run a **functional post-implementation** simulation and observe that the system operates correctly and that the filter exhibits the required behavior.



Finally, we run a **timing post-implementation** simulation, taking into account the delays introduced by the FPGA logic elements, and observe that the circuit once again exhibits the expected behavior.



To further verify the simulation results, the following Python script was developed:

```
def bin_to_int(b):
    """Unsigned conversion"""
    return int(b, 2)

def fir_with_ram_print(inputs, coeffs):
    n = len(coeffs)
    ram = [0.0] * n # shift register

    print("--- Demonstrating FIR Filter Usage ---\n")
    print(f"Initial filter coefficients (h): {coeffs}")
    print(f"Initial input array (X): {ram}\n")

    for i, x in enumerate(inputs):

        ram = [float(x)] + ram[:-1]

        y = sum(ram[j] * coeffs[j] for j in range(n))

        ram_str = " ".join(f"{v:.0f}." for v in ram)
        print(f"Iteration {i} --- | [{ram_str}] | {y:.4f}")

# -----
# VHDL INPUT (UNSIGNED)
# -----

vhdl_inputs = [
    "11010000", "11100111", "01000000", "11101001",
    "10100001", "00011000", "01000111", "10001100",
    "11110101", "11110111", "00101000", "11111000",
    "11110101", "01111100", "11001100", "00100100",
    "01101011", "11101010", "11001010", "11110101",
```

```

    "00000000", "00000000", "00000000", "00000000",
    "00000000", "00000000", "00000000", "00000000",
    "00000000", "00000000", "00000000", "00000000"
]

```

```
inputs = [bin_to_int(b) for b in vhdl_inputs]
```

```
coeffs = [1, 2, 3, 4, 5, 6, 7, 8]
```

```
fir_with_ram_print(inputs, coeffs)
```

The script produces the following results:

```

Iteration 0 --- | [208. 0. 0. 0. 0. 0. 0. 0.] | 208.0000
Iteration 1 --- | [231. 208. 0. 0. 0. 0. 0. 0.] | 647.0000
Iteration 2 --- | [64. 231. 208. 0. 0. 0. 0. 0.] | 1150.0000
Iteration 3 --- | [233. 64. 231. 208. 0. 0. 0. 0.] | 1886.0000
Iteration 4 --- | [161. 233. 64. 231. 208. 0. 0. 0.] | 2783.0000
Iteration 5 --- | [24. 161. 233. 64. 231. 208. 0. 0.] | 3704.0000
Iteration 6 --- | [71. 24. 161. 233. 64. 231. 208. 0.] | 4696.0000
Iteration 7 --- | [140. 71. 24. 161. 233. 64. 231. 208.] | 5828.0000
Iteration 8 --- | [245. 140. 71. 24. 161. 233. 64. 231.] | 5333.0000
Iteration 9 --- | [247. 245. 140. 71. 24. 161. 233. 64.] | 4670.0000
Iteration 10 --- | [40. 247. 245. 140. 71. 24. 161. 233.] | 5319.0000
Iteration 11 --- | [248. 40. 247. 245. 140. 71. 24. 161.] | 4631.0000
Iteration 12 --- | [245. 248. 40. 247. 245. 140. 71. 24.] | 4603.0000
Iteration 13 --- | [124. 245. 248. 40. 247. 245. 140. 71.] | 5771.0000
Iteration 14 --- | [204. 124. 245. 248. 40. 247. 245. 140.] | 6696.0000
Iteration 15 --- | [36. 204. 124. 245. 248. 40. 247. 245.] | 6965.0000
Iteration 16 --- | [107. 36. 204. 124. 245. 248. 40. 247.] | 6256.0000
Iteration 17 --- | [234. 107. 36. 204. 124. 245. 248. 40.] | 5518.0000
Iteration 18 --- | [202. 234. 107. 36. 204. 124. 245. 248.] | 6598.0000
Iteration 19 --- | [245. 202. 234. 107. 36. 204. 124. 245.] | 6011.0000
Iteration 20 --- | [0. 245. 202. 234. 107. 36. 204. 124.] | 5203.0000
Iteration 21 --- | [0. 0. 245. 202. 234. 107. 36. 204.] | 5239.0000
Iteration 22 --- | [0. 0. 0. 245. 202. 234. 107. 36.] | 4431.0000
Iteration 23 --- | [0. 0. 0. 0. 245. 202. 234. 107.] | 4931.0000
Iteration 24 --- | [0. 0. 0. 0. 0. 245. 202. 234.] | 4756.0000
Iteration 25 --- | [0. 0. 0. 0. 0. 0. 245. 202.] | 3331.0000
Iteration 26 --- | [0. 0. 0. 0. 0. 0. 0. 245.] | 1960.0000
Iteration 27 --- | [0. 0. 0. 0. 0. 0. 0. 0.] | 0.0000
Iteration 28 --- | [0. 0. 0. 0. 0. 0. 0. 0.] | 0.0000
Iteration 29 --- | [0. 0. 0. 0. 0. 0. 0. 0.] | 0.0000
Iteration 30 --- | [0. 0. 0. 0. 0. 0. 0. 0.] | 0.0000
Iteration 31 --- | [0. 0. 0. 0. 0. 0. 0. 0.] | 0.0000

```